

Raspberry Pi MQTT Buzzer Node

Master Cheat Sheet — Code, Configuration, Commands, and Home Assistant Automations

1. Python Script (The Engine)

File Location: /home/redwannabil/ha_buzzer_node.py

Purpose: Listens to Home Assistant and translates MQTT messages into physical buzzer sounds.

```
import time
import paho.mqtt.client as mqtt
from gpiozero import Buzzer

# =====
# CONFIGURATION
# =====
MQTT_SERVER = "192.168.0.40"
MQTT_PORT = 1883
MQTT_USER = "redwanmqtt" # Your verified HA Mosquitto user
MQTT_PASS = "abcd2005-" # Your HA Mosquitto password

COMMAND_TOPIC = "home/redwannabil/buzzer/cmd"
STATE_TOPIC = "home/redwannabil/buzzer/state"

# Initialize Buzzer on GPIO 17
buzzer = Buzzer(17)

# =====
# MQTT CALLBACKS
# =====
def on_connect(client, userdata, flags, reason_code, properties):
    print(f"[MQTT] Connected with result code {reason_code}")
    if reason_code == 0:
        client.subscribe(COMMAND_TOPIC)
        client.publish(STATE_TOPIC, "OFF", retain=True)
    else:
        print("Connection failed! Please check MQTT_USER and MQTT_PASS.")

def on_message(client, userdata, msg):
    payload = msg.payload.decode("utf-8")
    print(f"[CMD IN] {payload}")

    if payload == "ON":
        buzzer.on() # Solid continuous tone
        client.publish(STATE_TOPIC, "ON", retain=True)

    elif payload == "PULSE":
        buzzer.beep(0.5, 0.5) # Continuous pulsing siren
        client.publish(STATE_TOPIC, "PULSE", retain=True)

    elif payload == "OFF":
        buzzer.off() # Silence
        client.publish(STATE_TOPIC, "OFF", retain=True)

    elif payload == "BEEP":
        # Non-blocking quick beep for doors (doesn't change overall state)
        buzzer.beep(on_time=0.1, off_time=0.1, n=1, background=True)

# =====
# MAIN LOOP
# =====
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="PiServer_Buzzer_Node")
client.username_pw_set(MQTT_USER, MQTT_PASS)
client.on_connect = on_connect
client.on_message = on_message

print("[SYSTEM] Starting Pi MQTT Buzzer Service...")
client.connect(MQTT_SERVER, MQTT_PORT, 60)
client.loop_forever()
```

2. Linux Background Service (Systemd)

File Location: /etc/systemd/system/pi_buzzer.service

Purpose: Keeps the Python script running 24/7 in the background and restarts it if it crashes.

[Unit]

```
Description=Home Assistant MQTT Buzzer Node
After=network-online.target
```

```
[Service]
# The "-u" forces unbuffered output so you can see live logs!
ExecStart=/usr/bin/python3 -u /home/redwannabil/ha_buzzer_node.py
WorkingDirectory=/home/redwannabil
StandardOutput=inherit
StandardError=inherit
Restart=always
User=redwannabil
```

```
[Install]
WantedBy=multi-user.target
```

3. Essential Linux Management Commands

Run these commands in the Raspberry Pi terminal.

```
# Apply changes to the .service file
sudo systemctl daemon-reload

# Restart the service (apply Python code changes)
sudo systemctl restart pi_buzzer.service

# Start the service automatically on Raspberry Pi boot
sudo systemctl enable pi_buzzer.service

# View live real-time logs (crucial for debugging)
sudo journalctl -u pi_buzzer.service -f
```

4. Home Assistant Configuration (configuration.yaml)

File Location: /config/configuration.yaml (inside Home Assistant)

Purpose: Creates the manual switch.piserver_buzzer entity for the dashboard.

```
mqtt:
  switch:
    - name: "PiServer Buzzer"
      state_topic: "home/redwannabil/buzzer/state"
      command_topic: "home/redwannabil/buzzer/cmd"
      payload_on: "ON"      # Sends solid tone command
      payload_off: "OFF"    # Sends silence command
      state_on: "ON"
      state_off: "OFF"
      icon: mdi:alarm-bell
      unique_id: "piserver_buzzer_node"
```

5. Home Assistant Dashboard Card

Location: Dashboard → Edit → Add Card → Manual YAML

Purpose: A clean manual button to activate the solid emergency tone.

```
type: button
entity: switch.piserver_buzzer
name: PiServer Buzzer (Manual)
icon: mdi:bullhorn
show_state: true
tap_action:
  action: toggle
hold_action:
  action: none
```

6. Home Assistant Automations (Custom Sound Triggers)

Location: Settings → Automations → Create Automation → Edit in YAML

Purpose: Dynamically trigger different sounds such as PULSE or BEEP from smart-home sensors.

Example A: Trigger a short BEEP when a door opens

```
alias: "Door Opened -> Short Beep"
trigger:
  - platform: state
    entity_id: binary_sensor.front_door
    to: "on"
action:
```

```
- action: mqtt.publish
  data:
    topic: home/redwannabil/buzzer/cmd
    payload: "BEEP"
```

Example B: Trigger the PULSE siren if a Gas/Fire Sensor goes off

```
alias: "Fire Detected -> Pulse Siren"
trigger:
  - platform: state
    entity_id: binary_sensor.kitchen_gas_leak
    to: "on"
action:
  - action: mqtt.publish
    data:
      topic: home/redwannabil/buzzer/cmd
      payload: "PULSE"
```

Example C: Silence the Buzzer automatically when the Fire is cleared

```
alias: "Fire Cleared -> Turn Off Siren"
trigger:
  - platform: state
    entity_id: binary_sensor.kitchen_gas_leak
    to: "off"
action:
  - action: mqtt.publish
    data:
      topic: home/redwannabil/buzzer/cmd
      payload: "OFF"
```

MQTT Command	Buzzer Behavior	Reported State
ON	Solid continuous tone	ON
PULSE	Continuous pulsing siren	PULSE
OFF	Silence	OFF
BEEP	One short non-blocking beep	No state change

Note: The document preserves the supplied credentials and configuration exactly as provided. Treat MQTT credentials as sensitive and change them if this PDF is shared publicly.